



【Python/GRIB2】NOMADS (NOAA) の気象データから16時間後の気圧予報を取得する

2025/08/11に公開



Python



天気予報



気象データ



pygrid



気圧



Tech

課題

気象庁のオープンデータには気圧が含まれていないため、気圧予報ができない。NOMADS (NOAA) の気象予報データから世界の気象データを取得して、日本の気圧を抜き出す。

なぜ米NOAA?

日本の気象庁から民間化された「一般財団法人気象業務支援センター」から気象データをプロバインドしてもらうためには、お金がけっこうかかるため小規模でやるにはむずかしいです。情報提供負担金は以下参照です。

いちおう京都大学のKAGI21プロジェクトでGPV (Grid Point Value) のオリジンデータにアクセスできるようにしてくださっていますが、**教育研究機関向け** とのことなので個人系や商用系では使いづらい。

ここでは教育研究機関向けにデータを提供しています。企業活動等のためにデータを頻繁に必要とされる方は、気象業務支援センターからデータを直接購入し、データ提供スキーム全体の維持発展にご協力ください。

<https://database.rish.kyoto-u.ac.jp/arch/jmadata/>

また、気象庁のオープンデータには気圧予報がないので選択肢になりえませんでした。

NOMADS (NOAA)

アメリカ海洋大気庁 (National Oceanic and Atmospheric Administration) の各省庁の共同プロジェクトとして、天気予報から地球温暖化の研究まで共通の気象モデルでとりくむプロジェクトがあり、そのプロジェクト名が「NOMADS, NOAA Operational Model Archive and Distribution System」です。

一方で、過去のデータは、NOAA包括的地表観測データベース (NOAA-ISD, NOAA Integrated Surface Database) にあります。

<https://nomads.ncep.noaa.gov/> のサイトの、中段のできるだけ格子が細かい0.25 degreeのGFS (Global Forecast System) のモデルを選びます。

Pythonコード

`requests pandas pygrib` のみっつのパッケージを取り込んでいただき、実行したら、NOMADSのサーバーにアクセスして、指定した経緯度の気象データをJSONで15時間分の予報を返すコードです。

```
main.py
```

```
# -*- coding: utf-8 -*-
```

```
"""
```

NOAAのGFS(Global Forecast System)気象予報データサーバーから、最新の予報データを自動でダウンロードし、指定した地点の気温・湿度・気圧・降水強度を抽出するスクリプト。

主な機能:

- 最新の予報時刻を自動で判断し、データをダウンロードします。
- ダウンロードしたデータは、./assets/grib/ 以下に保存され、次回実行時に再利用されます。
- GRIB2形式のデータを解析し、扱いやすい辞書のリスト形式で気象データを返します。

```
"""
```

```
import json
```

```
import os
```

```
import sys
```

```
from datetime import datetime, timedelta, timezone
```

```
import pygrib
```

```
import requests
```

```
def fetch_latest_gfs_grib(force_download=False):
```

```
    """
```

最新のGFS気象予報データ(GRIB2形式)をダウンロードする。

対象のファイルが既に存在する場合は、そのパスを返しダウンロードをスキップする。

--force オプションが指定された場合は、既存のファイルがあっても強制的に再ダウンロードする。

Args:

force_download (bool): Trueの場合、強制的に再ダウンロードする。

Returns:

str | None: 成功した場合はダウンロードしたGRIB2ファイルのパス、失敗した場合

```
    """
```

GFSデータの配信遅延を考慮し、安全マージンとして6時間前の時刻を基準にする

```
base_time = datetime.now(timezone.utc) - timedelta(hours=6)
```

GFSは6時間ごと(00, 06, 12, 18 UTC)に発表されるため、直近の発表時刻を計算

```

hour = base_time.hour // 6 * 6
target_time = base_time.replace(hour=hour, minute=0, second=0, r

yyymmdd = target_time.strftime("%Y%m%d")
hh = target_time.strftime("%H")

# ダウンロードしたデータを保存するパス
output_grib_path = f"./assets/grib/gfs_latest_data_{yyymmdd}{hh}"

# 既存ファイルが見つかり、強制ダウンロードが指定されていない場合は再利用する
if not force_download and os.path.exists(output_grib_path):
    print(f"既存のファイルが見つかりました。再利用します: {output_grib_path}")
    return output_grib_path

# 保存先ディレクトリが存在しない場合は作成する
try:
    os.makedirs(os.path.dirname(output_grib_path), exist_ok=True)
except OSError as e:
    print(f"エラー: ディレクトリの作成に失敗しました。: {e}", file=sys.stderr)
    return None

# 強制ダウンロードの場合、古いファイルを削除
if os.path.exists(output_grib_path):
    os.remove(output_grib_path)

try:
    print(f"最新の気象データ(GFS)を取得します...")
    print(f"対象時刻 (UTC): {target_time.strftime('%Y-%m-%d %H:%M')}")

# NOAAのインデックスファイルで使われている正式名称で定義
elements_to_find = {
    "TMP": "TMP:2 m above ground", # 地上2m気温
    "RH": "RH:2 m above ground", # 地上2m相対湿度
    "PRMSL": "PRMSL:mean sea level", # 海面更正気圧
    "PRATE": "PRATE:surface", # 地表の降水強度
}

```

```

successful_hours = 0

# 0時間後から15時間後までの16時間分の予報を取得
for fh in range(16):
    forecast_hour = f"{fh:03d}"
    grib_filename = f"gfs.t{hh}z.pgrb2.0p25.f{forecast_hour}"
    base_url = f"https://nomads.ncep.noaa.gov/pub/data/nccf/"
    idx_url = f"{base_url}{grib_filename}.idx"
    grib_url = f"{base_url}{grib_filename}"

    print(f" {fh}時間後のデータを取得中...", end="", flush=True)

    # インデックスファイルを取得して、必要なデータの位置(バイト範囲)を特定する
    try:
        idx_response = requests.get(idx_url, timeout=15)
        idx_response.raise_for_status()
        idx_lines = idx_response.text.splitlines()
    except requests.exceptions.RequestException:
        print(" [失敗] インデックスファイルの取得に失敗。")
        continue

    byte_ranges = {}
    for i, line in enumerate(idx_lines):
        for key, name in elements_to_find.items():
            if name in line:
                if i + 1 >= len(idx_lines):
                    continue
                parts = line.split(":")
                next_line_parts = idx_lines[i + 1].split(":")
                start_byte = int(parts[1])
                end_byte = int(next_line_parts[1])
                byte_ranges[key] = (start_byte, end_byte)
                break

    # 主要3要素(気温、湿度、気圧)が見つからない場合はスキップ
    essential_keys = ["TMP", "RH", "PRMSL"]

```

```

        if not all(k in byte_ranges for k in essential_keys):
            print(f" [失敗] 主要データ(気温/湿度/気圧)が揃っていません。")
            continue

# HTTP Rangeリクエストで、巨大なGRIBファイルから必要な部分だけをダウン
with open(output_grib_path, "ab") as f:
    for key in sorted(byte_ranges.keys()):
        start_byte, end_byte = byte_ranges[key]
        headers = {"Range": f"bytes={start_byte}-{end_byte}"}
        grib_part_resp = requests.get(grib_url, headers=headers)
        grib_part_resp.raise_for_status()
        f.write(grib_part_resp.content)

    print(" [完了]")
    successful_hours += 1

if successful_hours == 0:
    print(
        "\nエラー: GRIB2ファイルの生成に失敗しました。ダウンロード可能なデータがありません。",
        file=sys.stderr,
    )
    return None

print(
    f"\nGRIB2ファイルの生成完了。({successful_hours}/16 時間分のデータ)",
)
return output_grib_path

except Exception as e:
    print(
        f"\nエラー: データのダウンロード中に予期せぬエラーが発生しました。: {e}",
        file=sys.stderr,
    )
    return None

```

```

def get_weather_data(grib_filepath: str):
    """
    GRIB2ファイルを解析し、指定地点の気象データを辞書のリストとして返す。

    Args:
        grib_filepath (str): 解析対象のGRIB2ファイルのパス。

    Returns:
        list[dict] | None: 抽出した気象データのリスト。各要素は1時間ごとの予報データ。
        エラーが発生した場合はNoneを返す。
    """
    # --- 定数設定 ---
    # データ抽出地点(例: 東京タワー)
    TARGET_LAT = 35.65858404079
    TARGET_LON = 139.74543164468
    # GFSの格子点間隔(0.25度)の半分。最近傍の格子点を見つけるために使用。
    GRID_HALF = 0.125
    # UTCから日本時間(JST)に変換するための時差
    JST_TIMEDELTA = timedelta(hours=9)

    try:
        # GRIBファイルを開き、すべてのメッセージを一度にリストとして読み込む
        with pygrib.open(grib_filepath) as gpv_file:
            all_messages = gpv_file.read()

            if not all_messages:
                return None

            weather_data_list = []
            # 0時間後から15時間後までの予報を処理
            for fh in range(16):
                # リスト内包表記とnext()を使い、各要素のメッセージを効率的に検索
                temp_msg = next(
                    (
                        m
                        for m in all_messages

```

```

        if m.name == "Apparent temperature" and m.forecast
    ),
    None,
)
rh_msg = next(
    (
        m
        for m in all_messages
        if m.name == "2 metre relative humidity" and m.t
    ),
    None,
)
pressure_msg = next(
    (
        m
        for m in all_messages
        if m.name == "Pressure reduced to MSL" and m.for
    ),
    None,
)
precip_msg = next(
    (
        m
        for m in all_messages
        if m.name == "Precipitation rate" and m.forecast
    ),
    None,
)

# 主要3要素(気温、湿度、気圧)が見つかった場合のみデータを採用
if temp_msg and rh_msg and pressure_msg:
    # 指定地点を含む矩形領域を指定し、最も近い格子点のデータを抽出
    temp_values, _, _ = temp_msg.data(
        lat1=TARGET_LAT - GRID_HALF,
        lat2=TARGET_LAT + GRID_HALF,
        lon1=TARGET_LON - GRID_HALF,

```



```

        lon2=TARGET_LON + GRID_HALF,
    )
    rh_values, _, _ = rh_msg.data(
        lat1=TARGET_LAT - GRID_HALF,
        lat2=TARGET_LAT + GRID_HALF,
        lon1=TARGET_LON - GRID_HALF,
        lon2=TARGET_LON + GRID_HALF,
    )
    pressure_values, _, _ = pressure_msg.data(
        lat1=TARGET_LAT - GRID_HALF,
        lat2=TARGET_LAT + GRID_HALF,
        lon1=TARGET_LON - GRID_HALF,
        lon2=TARGET_LON + GRID_HALF,
    )

    # 単位を変換し、扱いやすい数値にする
    temp_c = round(temp_values[0][0] - 273.15, 2) # ケル
    humidity = round(rh_values[0][0], 1)
    pressure_hpa = round(pressure_values[0][0] / 100.0,

    # 降水強度は、もしデータがあれば値を採用し、なければ0.0とする
    if precip_msg:
        precip_values, _, _ = precip_msg.data(
            lat1=TARGET_LAT - GRID_HALF,
            lat2=TARGET_LAT + GRID_HALF,
            lon1=TARGET_LON - GRID_HALF,
            lon2=TARGET_LON + GRID_HALF,
        )
        precipitation_mmhr = round(
            precip_values[0][0] * 3600, 2
        ) # kg m-2 s-1 -> mm/hr
    else:
        precipitation_mmhr = 0.0

    # 抽出したデータを辞書にまとめてリストに追加
    weather_data_list.append(

```

```

        {
            "valid_time_jst": (
                temp_msg.validDate + JST_TIMEDELTA
            ).isoformat(),
            "temperature_c": temp_c,
            "humidity_percent": humidity,
            "pressure_hpa": pressure_hpa,
            "precipitation_mmhr": precipitation_mmhr,
        }
    )

    return weather_data_list

except Exception as e:
    print(
        f"\nエラー: GRIBファイルの解析中に予期せぬエラーが発生しました。: {e}"
        file=sys.stderr,
    )
    return None

def main():
    """
    スクリプトのエントリーポイント。
    気象データを取得し、整形してコンソールに表示する。
    """
    # コマンドライン引数に--forceがあれば、強制的に再ダウンロードする
    force_download = "--force" in sys.argv

    # 1. GRIBファイルのパスを取得 (ダウンロードまたは再利用)
    grib_filepath = fetch_latest_gfs_grib(force_download)
    if not grib_filepath:
        sys.exit(1)

    # 2. GRIBファイルを解析して、気象データを辞書のリストとして取得
    weather_data = get_weather_data(grib_filepath)

```

```

# 3. 結果を表示
if weather_data:
    print("\n--- 取得した気象データ(辞書形式) ---")
    for data_point in weather_data:
        print(data_point)

    print("\n--- JSON形式での出力例 ---")
    print(json.dumps(weather_data, indent=2, ensure_ascii=False))

elif weather_data == []:
    print(
        "注意: 取得できたデータの中に、主要3要素(気温/湿度/気圧)が揃った時間
    )
else:
    print("処理に失敗しました。")

sys.exit(0)

if __name__ == "__main__":
    main()

```

以下のようなJSONが出来ます。

```

[
  {
    "valid_time_jst": "2025-08-11T09:00:00",
    "temperature_c": 32.65,
    "humidity_percent": 72.4,
    "pressure_hpa": 1005.75,
    "precipitation_mmhr": 0.24
  },
  {

```

```
"valid_time_jst": "2025-08-11T10:00:00",  
"temperature_c": 33.05,  
"humidity_percent": 72.4,  
"pressure_hpa": 1005.9,  
"precipitation_mmhr": 0.0  
},  
...
```

そもそのデータ構造的には、以下のようになっていて、CLWMR のような略字がキーマンのような働きをしています。

```
1:0:d=2025080300:PRMSL:mean sea level:anl:
2:1015227:d=2025080300:CLWMR:1 hybrid level:anl:
3:1152904:d=2025080300:ICMR:1 hybrid level:anl:
4:1364182:d=2025080300:RWMR:1 hybrid level:anl:
5:1660325:d=2025080300:SNMR:1 hybrid level:anl:
6:1741658:d=2025080300:GRLE:1 hybrid level:anl:
7:1780399:d=2025080300:REFD:1 hybrid level:anl:
8:2645404:d=2025080300:REFD:2 hybrid level:anl:
9:3510469:d=2025080300:REFC:entire atmosphere:anl:
10:4472308:d=2025080300:VIS:surface:anl:
11:5171129:d=2025080300:UGRD:planetary boundary layer:anl:
12:5786576:d=2025080300:VGRD:planetary boundary layer:anl:
13:6397063:d=2025080300:VRATE:planetary boundary layer:anl:
14:6713582:d=2025080300:GUST:surface:anl:
15:7350968:d=2025080300:HGT:0.01 mb:anl:
16:8035963:d=2025080300:TMP:0.01 mb:anl:
17:8443683:d=2025080300:RH:0.01 mb:anl:
18:8460181:d=2025080300:SPFH:0.01 mb:anl:
19:9224732:d=2025080300:VVEL:0.01 mb:anl:
20:10284876:d=2025080300:DZDT:0.01 mb:anl:
21:11274543:d=2025080300:UGRD:0.01 mb:anl:
22:11679454:d=2025080300:VGRD:0.01 mb:anl:
23:12043731:d=2025080300:ABSV:0.01 mb:anl:
24:12591194:d=2025080300:O3MR:0.01 mb:anl:
25:13194443:d=2025080300:HGT:0.02 mb:anl:
...
...

https://nomads.ncep.noaa.gov/pub/data/nccf/com/gfs/prod/gdas.2025080
```

勉強になる記事



Discussion



ログインするとコメントできます

Login

Read next



Kaggle Notebook で始める CUDAプログラミング入門



kaerururu 9時間前 ♡ 1



[Python]Hono CLIを参考にCLI型コーディングエージェントでドキュメント参照できるようにした(Click)



黒井樹 8時間前 ♡ 1



[Python 100日チャレンジ] Day 2 - Pythonのインストール



YonYonWare 18時間前

初めてのRAG(langchain & ollama)



takada 21時間前

Pythonで勉強タイマーを作ってみた(初学者の試行錯誤ログ)



Volx 23時間前

【機械学習入門】1つのモデルから複数の推論を得るには【マルチタスク学習】



myojin in ファーストループテクノロジー株式会社

1日前